

THEORY AND METHODS A REVIEW OF KERNEL FISHER DISCRIMINANT ANALYSIS FOR STATISTICAL CLASSIFICATION

N. Louw and S.J. Steel

Department of Statistics and Actuarial Science, University of Stellenbosch,
Private Bag X1, Matieland, 7602, South Africa

Key words: Feature space; generalization error; input space; kernel methods; linear discriminant analysis.

Summary: Discriminant analysis is a well-known and frequently used class of statistical classification procedures. A recent addition to this class is kernel Fisher discriminant analysis (KFDA). This procedure, originally proposed in the machine learning literature, has so far been applied mainly in areas such as artificial intelligence, machine learning and pattern recognition, with little exposure in statistics. Given its excellent performance in difficult classification problems, it is important for statisticians working in classification to become acquainted with KFDA. In this paper we discuss the rationale behind KFDA, comment on practical aspects which have to be dealt with during its implementation, describe a new proposal for determining the value of the Gaussian kernel parameter, and report the results of a simulation study comparing KFDA and linear discriminant analysis.

1. Introduction

The introduction of the support vector machine by Boser et al. (1992) initiated a period of rapid development in the theory and practice of kernel methods. These developments have mainly been in areas such as artificial intelligence, machine learning, and pattern recognition, where kernel methods have been widely and successfully applied. Examples of kernel methods

STMA (2005) SUBJECT CLASSIFICATION: 06:050(06:900)

are support vector machines, kernel Fisher discriminant analysis, kernel principal component analysis, and kernel logistic regression (see Herbrich, 2001, Schölkopf and Smola, 2002, Shawe-Taylor and Cristianini, 2004, and Kroon and Omlin, 2004a, 2004b, for recent comprehensive discussions). Despite the fact that kernel methods frequently deliver state-of-the-art solutions to regression and classification problems, these methods are still largely unfamiliar amongst statisticians. This is reflected in the absence of modules for implementing kernel methods from most of the well-known statistical software packages. Our purpose in this review paper is to introduce a kernel method which performs excellently in classification problems to a wider statistical audience. *Kernel Fisher discriminant analysis* (KFDA), although less well-known than support vector machines, provides classification performance on a comparable level. In addition, in contrast to support vector classifiers, KFDA offers the possibility of quantifying the posterior probabilities of group membership. All in all, KFDA deserves serious consideration as a procedure for statistical classification, and it offers many interesting opportunities for further research.

In Section 2 we briefly discuss the rationale behind kernel methods in general and KFDA in particular. Section 3 contains a discussion of several further aspects regarding KFDA, an algorithm for implementing the procedure, comments on practical problems encountered when applying KFDA, and a brief description of a new technique for solving one of these problems. We also refer in this section to software options for KFDA. In Section 4 we report the results of an empirical study to evaluate KFDA in terms of its error rate compared to that of linear discriminant analysis. Section 5 contains remarks on applications and a discussion of several directions for future research.

2. Kernel methods and KFDA

Consider the following generic two-group classification problem. We observe a binary response variable $Y \in \{-1, +1\}$, together with classification or input variables $X_1, X_2, \dots, X_p$. These variables are observed for $n = n_1 + n_2$ sample cases, with the first n_1 cases from population 1 (where we have $Y = +1$) and the remaining n_2 cases from population 2. The resulting *training* data set is denoted by $\mathcal{T} = \{(\mathbf{x}_i, y_i), i = 1, 2, \dots, n\}$. Here, $\mathbf{x}_i$ is a p -component vector containing the values of $X_1, X_2, \dots, X_p$ for case i in the sample. These vectors are often referred to as *patterns* in the machine learning literature; we will also use the term *input vectors*. The objective is to use $\mathcal{T}$ to find a function, $f(\mathbf{x})$, called a *discriminant function*, so that $\text{sign}\{f(\mathbf{x})\}$ can be used to assign a new case with observed values of the classification variables in the vector $\mathbf{x}$ to one of the two populations. We will refer to $\text{sign}\{f(\mathbf{x})\}$ as a *classifier*. Linear discriminant analysis (LDA) is one of the most frequently used statistical procedures for solving such classification problems, being especially successful in cases where $X_1, X_2, \dots, X_p$ have approximate normal distributions. Attempts to broaden the applicability of LDA have led to several extensions of the basic technique, yielding for example quadratic discriminant analysis, flexible discriminant analysis, penalized discriminant analysis, and regularized discriminant analysis (see Hastie et al., 2001, for a discussion of these techniques).

KFDA was introduced by Mika et al. (1999). Consider the classifier $\text{sign}\{f(\mathbf{x})\}$, and suppose $f(\mathbf{x})$ is a linear function, i.e. $f(\mathbf{x}) = \beta + \langle \boldsymbol{\nu}, \mathbf{x} \rangle$. In this expression, the scalar β and the vector $\boldsymbol{\nu}$ are quantities which have to be determined from the sample data, while $\langle \boldsymbol{\nu}, \mathbf{x} \rangle = \sum \nu_i x_i$ denotes the inner product between $\boldsymbol{\nu}$ and $\mathbf{x}$. There are several advantages if $f(\mathbf{x})$ is a linear function: the decision boundary $\{\mathbf{x} : f(\mathbf{x}) = 0\}$ is a geometrically

simple set, viz. a hyperplane in $\mathfrak{R}^p$, and there are only $p + 1$ parameters which have to be estimated from the data, considerably reducing the danger of overfitting, especially in cases where p is large compared to the sample size n . This latter advantage frequently translates into good test error behaviour of the classifier.

LDA is arguably the best known example of a linear classification technique. The LDA classifier is given by $\text{sign}\{\beta_{LDA} + \langle \nu_{LDA}, \mathbf{x} \rangle\}$, where ν_{LDA} maximizes the ratio

$$J(\nu) = \frac{\nu' B \nu}{\nu' W \nu} = \frac{\langle B \nu, \nu \rangle}{\langle W \nu, \nu \rangle}. \quad (2.1)$$

In (2.1), $B = (\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)'$ is the between class scatter matrix, while W is the pooled sample covariance matrix defined by the expression $(n_1 + n_2 - 2)W = \sum_{i=1}^{n_1} (\mathbf{x}_i - \bar{\mathbf{x}}_1)(\mathbf{x}_i - \bar{\mathbf{x}}_1)' + \sum_{i=n_1+1}^n (\mathbf{x}_i - \bar{\mathbf{x}}_2)(\mathbf{x}_i - \bar{\mathbf{x}}_2)'$, with $\bar{\mathbf{x}}_1$ and $\bar{\mathbf{x}}_2$ the sample mean vectors. We make the usual assumption that W is a non-singular matrix. A well-known extension of the Cauchy-Schwarz inequality implies that the ν maximizing (2.1) is proportional to $W^{-1}(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)$, i.e. $\nu_{LDA} = cW^{-1}(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)$. We can take the proportionality constant to be 1, giving $\nu_{LDA} = W^{-1}(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)$. The intercept traditionally used in LDA is given by $\beta_{LDA} = \frac{1}{2}(\bar{\mathbf{x}}_2' W^{-1} \bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1' W^{-1} \bar{\mathbf{x}}_1) + \log\left(\frac{n_1}{n_2}\right)$, and the LDA rule to classify a new case with input vector $\mathbf{x}$ is therefore

$$\text{sign} \left\{ \frac{1}{2} (\bar{\mathbf{x}}_2' W^{-1} \bar{\mathbf{x}}_2 - \bar{\mathbf{x}}_1' W^{-1} \bar{\mathbf{x}}_1) + \log\left(\frac{n_1}{n_2}\right) + \langle W^{-1}(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2), \mathbf{x} \rangle \right\}. \quad (2.2)$$

Despite the advantages offered by a linear $f(\mathbf{x})$, there are many cases where these simple classifiers are inadequate and a more sophisticated approach is required. Using a kernel method such as KFDA is one possibility. To explain the basic idea of KFDA, consider the input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$,

and the input space $\mathfrak{R}^p$. Conceptually KFDA entails transformation of the $\mathbf{x}_i \in \mathfrak{R}^p$ to a high dimensional *feature space*, followed by application of the usual LDA procedure in feature space. Although the procedure resulting from this will be linear in feature space, it corresponds to a non-linear technique in input space. Let $\Phi(\cdot)$ denote the transformation from input to feature space, with $\mathbf{z} = \Phi(\mathbf{x})$ a transformed input vector, also called a feature vector. Since the dimension of the feature space is typically very high and can even be infinite, performing calculations in this space is problematic. In KFDA, as in other kernel based procedures, we overcome this difficulty as follows. Consider functions $K : \mathfrak{R}^p \times \mathfrak{R}^p \rightarrow \mathfrak{R}$ satisfying the following two conditions: (i) for every value of $n \in \{1, 2, \dots\}$, and all real vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, the $n \times n$ matrix with ij^{th} element $K(\mathbf{x}_i, \mathbf{x}_j)$ is positive semi-definite; (ii) K is a symmetric function, i.e. $K(\mathbf{x}_i, \mathbf{x}_j) = K(\mathbf{x}_j, \mathbf{x}_i)$. These functions are called (positive semi-definite) *kernels*. Given a kernel K it is possible to construct a feature space consisting of vectors $\mathbf{z} = \Phi(\mathbf{x})$ so that the value of an inner product in feature space can be obtained by simply calculating the value of the kernel, i.e. $\langle \Phi(\mathbf{x}), \Phi(\tilde{\mathbf{x}}) \rangle = K(\mathbf{x}, \tilde{\mathbf{x}})$. This is a consequence of Mercer's theorem, a result from the theory of reproducing kernel Hilbert spaces. The important implication of this result is that if the algorithm which we apply to construct a classifier in a high-dimensional feature space has the property that feature vectors $\mathbf{z} = \Phi(\mathbf{x})$ appear in the algorithm only in the form of inner products, the *kernel trick*, whereby we substitute $K(\mathbf{x}, \tilde{\mathbf{x}})$ for $\langle \Phi(\mathbf{x}), \Phi(\tilde{\mathbf{x}}) \rangle$, can be used to obviate explicit specification of $\Phi(\cdot)$ as well as calculations in feature space. In practice this is implemented by specifying and using a kernel without ever considering the implied feature mapping $\Phi(\cdot)$ or the space which it induces.

Two popular kernels are the *polynomial kernel*, $K(x_i, x_j) = (\langle x_i, x_j \rangle + \theta)^d$, with θ a non-negative real quantity and d a positive integer, usually 2 or 3, and the *Gaussian radial basis function (RBF) kernel*, $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, with γ a positive real number. Here, $\|x\|^2 = \langle x, x \rangle = \sum_{i=1}^p x_i^2$ denotes the squared Euclidean norm. The quantities θ , d and γ are called *hyperparameters*, and their values have to be specified by the user, or determined from the data. We will return to this point in Section 3. For more detailed discussions of these and other kernel functions, see Herbrich (2001), Schölkopf and Smola (2002), and Shawe-Taylor and Cristianini (2004).

We proceed with a more detailed exposition of KFDA. Let $\bar{z}_1$ and $\bar{z}_2$ be the mean vectors for the two groups of feature vectors $z_1, z_2, \dots, z_n$ corresponding to the training data input vectors, and suppose $B_z = (\bar{z}_1 - \bar{z}_2)(\bar{z}_1 - \bar{z}_2)'$, and $(n_1 + n_2 - 2)W_z = \sum_{i=1}^{n_1} (z_i - \bar{z}_1)(z_i - \bar{z}_1)' + \sum_{i=n_1+1}^n (z_i - \bar{z}_2)(z_i - \bar{z}_2)'$. To find Fisher's linear discriminant in feature space, the ratio

$$J(u) = \frac{u' B_z u}{u' W_z u} = \frac{\langle B_z u, u \rangle}{\langle W_z u, u \rangle} \quad (2.3)$$

has to be maximized. Assuming W_z to be non-singular, the solution $\tilde{u} = W_z^{-1}(\bar{z}_1 - \bar{z}_2)$ could in principle be used to construct a classification rule

$$\text{sign}\{b + \langle \tilde{u}, z \rangle\} \quad (2.4)$$

in feature space, but the high dimensionality of this space generally makes such a process impracticable. We require a formulation of the problem allowing us to use the kernel trick. Consider therefore the linear space $\mathcal{D}$ spanned by $z_1, z_2, \dots, z_n$. Any vector u in feature space can be written in the form $u = u_1 + u_2$, where $u_1 \in \mathcal{D}$ and $u_2 \in \mathcal{D}^\perp$, the orthogonal complement of $\mathcal{D}$. It readily follows that $\langle Au, u \rangle = \langle Au_1, u_1 \rangle$ for any symmetric

matrix A whose rows (or columns) belong to $\mathcal{D}$. It can easily be verified that the rows of the symmetric matrices W_z and B_z belong to $\mathcal{D}$, and therefore $\langle B_z u, u \rangle = \langle B_z u_1, u_1 \rangle$ and $\langle W_z u, u \rangle = \langle W_z u_1, u_1 \rangle$ for any u in feature space. Maximizing (2.3) is therefore equivalent to maximizing $J(u_1) = \frac{u_1' B_z u_1}{u_1' W_z u_1} = \frac{\langle B_z u_1, u_1 \rangle}{\langle W_z u_1, u_1 \rangle}$ over $u_1 \in \mathcal{D}$, and it is therefore sufficient to restrict attention to vectors u belonging to $\mathcal{D}$, i.e. vectors which can be written in the form $u = \sum_{i=1}^n \alpha_i z_i = \sum_{i=1}^n \alpha_i \Phi(x_i)$ for given scalars $\alpha_1, \alpha_2, \dots, \alpha_n$. Using this result and the kernel trick we are able to write

$$\langle u, \bar{z}_1 \rangle = \frac{1}{n_1} \sum_{i=1}^n \sum_{j=1}^{n_1} \alpha_i \langle \Phi(x_i), \Phi(x_j) \rangle = \frac{1}{n_1} \sum_{i=1}^n \sum_{j=1}^{n_1} \alpha_i K(x_i, x_j) = \langle \alpha, m_1 \rangle,$$

with the n elements of m_1 given by $\frac{1}{n_1} \sum_{j=1}^{n_1} K(x_i, x_j)$, and with α an n -vector with elements $\alpha_1, \alpha_2, \dots, \alpha_n$. Similarly, we find that $\langle u, \bar{z}_2 \rangle = \langle \alpha, m_2 \rangle$. If we now define $M = (m_1 - m_2)(m_1 - m_2)'$, it readily follows that the numerator of $J(u)$ in (2.3) is given by $u' B_z u = \alpha' M \alpha$. One can also obtain the denominator of (2.3) in a tractable form. Define an $n \times n_1$ kernel matrix K_1 for class 1 to be the matrix with elements $K(x_i, x_j)$, $i = 1, \dots, n$; $j = 1, \dots, n_1$, and write I_{n_1} for the $n_1 \times n_1$ matrix with all elements equal to $1/n_1$. Define K_2 and I_{n_2} similarly. If we now set $N = \sum_{j=1}^2 K_j (I - I_{n_j}) K_j'$, it follows that the denominator of $J(u)$ in (2.3) can be written as $u' W_z u = \alpha' N \alpha$. We conclude that maximizing (2.3) is equivalent to maximizing the ratio

$$J(\alpha) = \frac{\alpha' M \alpha}{\alpha' N \alpha} = \frac{M \alpha, \alpha}{N \alpha, \alpha}. \quad (2.5)$$

While maximization of (2.3) will generally be impossible (because it has to be done over possibly infinite dimensional vectors u in feature space), maximization of (2.5) is in this regard simple, since it only involves vectors $\alpha \in \mathfrak{R}^n$. Suppose $\tilde{\alpha}$ maximizes (2.5), and let $\tilde{u} = \sum_{i=1}^n \tilde{\alpha}_i \Phi(x_i)$. Since

$\langle \tilde{\mathbf{u}}, \mathbf{z} \rangle = \sum_{i=1}^n \tilde{\alpha}_i \langle \Phi(\mathbf{x}_i), \Phi(\mathbf{x}) \rangle = \sum_{i=1}^n \tilde{\alpha}_i K(\mathbf{x}_i, \mathbf{x})$, the LDA classification rule (2.4) in feature space becomes

$$\text{sign}\{b + \langle \tilde{\mathbf{u}}, \mathbf{z} \rangle\} = \text{sign}\left\{b + \sum_{i=1}^n \tilde{\alpha}_i K(\mathbf{x}_i, \mathbf{x})\right\} \quad (2.6)$$

which is a practically useful result once the coefficients $\tilde{\alpha}_1, \tilde{\alpha}_2, \dots, \tilde{\alpha}_n$ and the intercept b have been determined. We will refer to (2.6) as the KFDA classification rule. Notice in (2.6) how application of the kernel trick once again obviates explicit specification or use of the feature mapping $\Phi(\cdot)$.

The above approach does, however, leave us with one problem: the matrix N is singular and consequently we cannot find the maximizing $\tilde{\alpha}$ by simply calculating $\tilde{\alpha} = N^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$. Mika et al. (1999) propose and motivate the use of regularization to overcome this difficulty. In the present context regularization entails replacing N by a matrix $N_\lambda = N + \lambda I$, for some (small) positive scalar λ . This yields a solution $\tilde{\alpha} = N_\lambda^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$, depending on λ , which can be used in (2.6). Obviously the hyperparameter λ has to be specified, and this will receive attention in Section 3.

The intercept b in (2.6) can be specified in different ways. A popular choice is to use $b = \frac{1}{2}(\mathbf{m}_2' N_\lambda^{-1} \mathbf{m}_2 - \mathbf{m}_1' N_\lambda^{-1} \mathbf{m}_1) + \log\left(\frac{n_1}{n_2}\right) = -\frac{1}{2}\tilde{\alpha}'(\mathbf{m}_1 + \mathbf{m}_2) + \log\left(\frac{n_1}{n_2}\right)$, which shows a clear similarity to the intercept used in LDA.

There are also other approaches which may be used to derive the KFDA classification rule. Mika et al. (2001) show how this can be done by solving a convex quadratic optimization problem, while Suykens et al. (2002) exploit a link between least squares support vector machines and LDA in feature space. Finally, Shawe-Taylor and Cristianini (2004) present an argument which enables one to find the KFDA classification function by solving a ridge-like problem in feature space. In our empirical work we used the

approach which was presented above, based on regularizing the matrix N . The properties and problematic issues discussed in the next section apply equally well to the other approaches.

3. Further aspects regarding KFDA

Consider a new case with observed values of the classification variables in a vector x . Using KFDA, this case can be classified into either population 1 or population 2 by applying (2.6). Let $p_1(x)$ denote the posterior probability of this case coming from population 1, with $p_2(x) = 1 - p_1(x)$ the corresponding posterior probability of coming from population 2. An important advantage of KFDA compared to support vector machines is that the former procedure allows one to estimate these posterior probabilities. Consider in this regard the quantities $r_i = \sum_{j=1}^n \tilde{\alpha}_j K(x_i, x_j)$, $i = 1, 2, \dots, n$. These are simply the projections of the training input vectors onto the direction in feature space determined by the kernel Fisher discriminant (the vector $\tilde{\alpha}$). We can use $r_1, r_2, \dots, r_{n_1}$ and $r_{n_1+1}, r_{n_1+2}, \dots, r_{n_1+n_2}$ to estimate a probability density function for the r_i values from population 1 and population 2 respectively. Denote these estimated densities by $\hat{f}_1$ and $\hat{f}_2$ respectively. The posterior probabilities $p_1(x)$ and $p_2(x)$ are then estimated in the usual way by applying Bayes' theorem, i.e. we take $\hat{p}_1(x) = \frac{\pi_1 \hat{f}_1(x)}{\pi_1 \hat{f}_1(x) + \pi_2 \hat{f}_2(x)}$, where π_1 and π_2 are the population prior probabilities (estimated for example by n_1/n and n_2/n respectively). According to Schölkopf and Smola (2002, p. 464) there is empirical evidence suggesting that we may take $\hat{f}_1$ and $\hat{f}_2$ to be normal densities, in which case estimating these densities only involve estimation of the normal distribution parameters.

There are several algorithms for implementing KFDA. The exposition in Section 2 leads to the following straightforward steps (see also Herbrich, 2001).

1. (a) Select a kernel function. We will use the Gaussian kernel given by $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2\right)$ in the remainder of our discussion.
- (b) Standardize the observations on each of the input variables to have zero mean and unit standard deviation. This is standard practice in the application of kernel methods, and often leads to improved results (cf. Schölkopf and Smola, 2002).
- (c) Decide on values for the kernel hyperparameters, or determine these values from the available training data. Below, we describe two procedures which can be used to find a value for γ in the Gaussian kernel from the available training data: cross-validation, and a procedure proposed by Steel and Louw (2004).
- (d) Determine a value for the regularization parameter λ to be used in $N_\lambda = N + \lambda I$. We indicate below how cross-validation may be used for this purpose as well.
- (e) Calculate the vectors $\mathbf{m}_1$ and $\mathbf{m}_2$, and the matrices N and N_λ . Use a linear algebra software routine to solve the set of equations $N_\lambda \alpha = (\mathbf{m}_1 - \mathbf{m}_2)$, thereby obtaining the coefficients $\tilde{\alpha}_1, \tilde{\alpha}_2, \dots, \tilde{\alpha}_n$ required in (2.6).
- (f) Calculate the intercept $b = -\frac{1}{2} \tilde{\alpha}'(\mathbf{m}_1 + \mathbf{m}_2) + \log\left(\frac{n_1}{n_2}\right)$. The KFDA classification rule (2.6) can now be used to classify new cases (which also have to be standardized) into one of the two populations.

Steps (a)–(f) can be implemented in any programming language environment. The remaining problem is determining values for the hyperparameters γ and λ . It should be noted that the generalization performance of the KFDA classification rule, i.e. its ability to classify new cases into the correct population, is typically quite sensitive to the values used for these hyperparameters. The method most commonly recommended in the literature for determining values for γ and λ is k -fold cross-validation. In

this we construct a grid of (γ, λ) pairs by combining each of r values for γ , say $\gamma_1, \gamma_2, \dots, \gamma_r$, with each of s values for λ , say $\lambda_1, \lambda_2, \dots, \lambda_s$, yielding $t = rs$ such pairs. In this regard a log-scale is frequently used for both γ and λ . The training data are divided into k non-overlapping parts, i.e. $\mathcal{T} = \mathcal{T}_1 \cup \mathcal{T}_2 \cup \dots \cup \mathcal{T}_k$. We first perform a KFDA on the available data excluding the subset $\mathcal{T}_1$, obtaining the classification rule in (2.6) for each of the t (γ, λ) pairs. We now treat the data in $\mathcal{T}_1$ as new cases which have to be classified, and apply each of the KFDA classification rules to this data, yielding t error or misclassification rates on $\mathcal{T}_1$. This process is performed k times, excluding the subsets $\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_k$ in turn, and each time calculating an error rate for the KFDA classification rule corresponding to each of the t (γ, λ) pairs. The (γ, λ) pair giving the lowest average error rate over all k repetitions provides us with values for γ and λ which may be used in steps (e) and (f) above.

This cross-validation procedure for determining values for γ and λ can be extremely time consuming, especially if we have to search over fairly wide grids. A much faster method which can be used for determining a value for γ is proposed in Steel and Louw (2004), who argue as follows. Transformation of the input vectors, $\mathbf{x}_i$, to high dimensional feature vectors, $\mathbf{z}_i$, is a crucial component of any kernel method. Underlying this transformation in classification applications is the belief that there will be a (strong) relationship between the grouping of the $\mathbf{z}_i$'s and the class labels, y_i (see Cristianini et al., 2002, for a more detailed discussion). The closeness or similarity of two feature vectors, $\mathbf{z}_i$ and $\mathbf{z}_j$, is reflected in the value of the inner product $\langle \mathbf{z}_i, \mathbf{z}_j \rangle$. Since $\langle \mathbf{z}_i, \mathbf{z}_j \rangle = K(\mathbf{x}_i, \mathbf{x}_j)$, it seems an acceptable heuristic to select the parameters of the kernel function in such a way that $K(\mathbf{x}_i, \mathbf{x}_j)$ tends to assume a "large" value whenever $\mathbf{x}_i$ and $\mathbf{x}_j$ correspond to sample cases from the same group, and a "small" value whenever these input vectors

correspond to different groups. For the Gaussian kernel, $0 < K(\mathbf{x}_i, \mathbf{x}_j) \leq 1$, i.e. if we put $h_{ij} = 2K(\mathbf{x}_i, \mathbf{x}_j) - 1$ (suppressing the dependence of this quantity on γ), we have $-1 < h_{ij} \leq 1$. If we regard $K(\mathbf{x}_i, \mathbf{x}_j) < 0.5$ as a “small” value of the kernel, and $K(\mathbf{x}_i, \mathbf{x}_j) > 0.5$ as a “large” value, this translates into $h_{ij} < 0$ and $h_{ij} > 0$ respectively. This suggests the following procedure for determining a value for γ . Let $\mathcal{J}_1 = \{1, 2, \dots, n_1\}$ be the set of indices corresponding to the sample cases from population 1, and $\mathcal{J}_2 = \{n_1 + 1, n_1 + 2, \dots, n_1 + n_2\}$ the set of indices for the sample cases from population 2. Define the criterion

$$A(\gamma) = \sum_{\substack{i, j \in \mathcal{J}_1 \\ i > j}} I(h_{ij} > 0) + \sum_{\substack{i, j \in \mathcal{J}_2 \\ i > j}} I(h_{ij} > 0) + \sum_{i \in \mathcal{J}_1, j \in \mathcal{J}_2} I(h_{ij} < 0). \quad (3.1)$$

Then $A(\gamma)$ is simply the total number of sample cases for which h_{ij} has the “correct” sign, where a correct sign of h_{ij} should be interpreted as $h_{ij} > 0$ whenever the indices i and j correspond to the same population, and $h_{ij} < 0$ whenever these indices correspond to different populations. The idea is to find γ to maximize $A(\gamma)$, and Steel and Louw (2004) describe a very simple and fast algorithm for accomplishing this objective.

Steel and Louw (2004) recommend that γ and λ should be determined by first maximizing $A(\gamma)$, followed by cross-validation along a 1-dimensional grid to decide on a value for λ . In the cross-validation part it is usually sufficient to consider approximately 20 values of λ , spaced on a log-scale over an interval of the form $[e^{-10}, e^{10}]$. Steel and Louw (2004) show that this is much faster than using cross-validation over a 2-dimensional grid of (γ, λ) pairs, and present empirical evidence suggesting that the generalization ability of the classification rule based on their proposal is comparable to that of the rule where γ and λ are found by means of cross-validation over a 2-dimensional grid.

Turning to software for implementing KFDA, it is straightforward to program the algorithm described earlier in this section using a suitable programming language. To our knowledge KFDA has not yet appeared in any of the large standard statistical software packages. Routines for this purpose have, however, been made available on the Internet, typically in *Matlab* and in *R*. A good *Matlab/C* option is the toolbox *LS-SVMlab1.5* (see Suykens et al., 2004, in this regard). Useful websites for keeping up to date with recent developments in kernel methods, including new software, are: <http://www.kernel-machines.org> and <http://www.learning-kernel-classifiers.org>.

4. Monte Carlo simulation study

Since its introduction in Mika et al. (1999), KFDA has been applied mainly in areas such as machine learning, artificial intelligence and pattern recognition. Several studies have been undertaken to investigate the generalization performance of KFDA in such applications (see for example Mika et al., 2001). These studies were mainly focused on benchmark data sets, and no systematic simulation investigation into the classification ability of the KFDA rule has as yet been reported in the literature. We therefore undertook a fairly extensive Monte Carlo simulation study to compare the classification performance of LDA and KFDA in several different contexts. In this study we used a 6-factor experimental design. The factors and their levels were:

- The distribution from which the input vectors $\mathbf{x}_i$ were generated. We used normal and lognormal distributions.
- The training sample size. Three levels were used, viz. $n_1 = n_2 = 50$ (small samples), $n_1 = n_2 = 100$ (large samples) and $n_1 = 25, n_2 = 75$ (mixed samples).
- The dimension of the input vectors. We used two levels, viz. $p = 5$ and $p = 10$.

- The covariance structure for generating the training data. Two levels were used, viz. $\Sigma = I$, the identity matrix, giving uncorrelated input variables, and $\Sigma = \Sigma^{(0.5)}$, representing the equi-correlated case with diagonal elements of Σ , $\sigma_{ii} = 1$, $i = 1, \dots, p$, and off-diagonal elements $\sigma_{ij} = 0.5$, $\forall i \neq j$, $i, j = 1, \dots, p$.
- The parameters with respect to which the two populations differed. We investigated two cases: firstly, the case where the populations have identical covariance matrices, i.e. $\Sigma_1 = \Sigma_2$, but differ with respect to mean vectors, i.e. $\mu_1 \neq \mu_2$, and secondly, the case where the populations have identical mean vectors, but different covariance matrices.
- The separation between the two populations. This was systematically varied by increasing the difference between μ_1 and μ_2 , or between Σ_1 and Σ_2 . The top half of each of Table 1 and 2 is devoted to cases described by “Group mean vectors differ”. In these cases we set $\mu_1 = 0$ and systematically increased the values of μ_{2j} , thereby increasing the separation between the two populations. Identical covariance matrices were used, i.e. $\Sigma_1 = \Sigma_2 = \Sigma$. We quantified the separation between the two populations in terms of Mahalanobis distance, i.e. $\Delta^2 = (\mu_1 - \mu_2)' \Sigma^{-1} (\mu_1 - \mu_2)$. In the columns headed “Spacing 1”, we had $\Delta^2 = 1$ in Table 1 and 2. In the columns with heading “Spacing 2”, we had $\Delta^2 = 8$ in Table 1 and $\Delta^2 = 4$ in Table 2. The bottom half of each of Table 1 and 2 is devoted to cases described by “Group covariance matrices differ”. Here we set $\mu_1 = \mu_2 = 0$, so that the two populations do not differ with respect to location. We used a covariance matrix Σ_1 for population 1, and obtained Σ_2 from Σ_1 by multiplying the diagonal elements of the latter by a scale factor. In the columns headed “Spacing 1”, we used a scale factor equal to 2, while a scale factor of 6 was used in the columns with heading “Spacing 2”.

A total of 96 scenarios were therefore investigated. In each scenario we generated training data from the appropriate underlying distribution, and constructed the LDA classification rule (2.2), as well as the KFDA classification rule (2.6). For the KFDA rule we used the proposal by Steel and Louw (2004) to determine the value of γ in the Gaussian kernel, combined with 10-fold cross-validation to find a value for the regularization

Table 1. Average simulated test errors for LDA and KFDA in the case of normal populations

				Spacing 1		Spacing 2	
				LDA	KFDA	LDA	KFDA
Group mean vectors differ	$p = 5$	$n_1 = 50$	$\rho = 0$	0.325	0.340	0.086	0.086
		$n_2 = 50$	$\rho = 0.5$	0.326	0.320	0.087	0.084
		$n_1 = 100$	$\rho = 0$	0.318	0.326	0.082	0.083
		$n_2 = 100$	$\rho = 0.5$	0.317	0.315	0.083	0.082
		$n_1 = 25$	$\rho = 0$	0.259	0.246	0.076	0.075
		$n_2 = 75$	$\rho = 0.5$	0.258	0.241	0.076	0.070
	$p = 10$	$n_1 = 50$	$\rho = 0$	0.343	0.352	0.095	0.090
		$n_2 = 50$	$\rho = 0.5$	0.342	0.322	0.096	0.084
		$n_1 = 100$	$\rho = 0$	0.327	0.333	0.086	0.085
		$n_2 = 100$	$\rho = 0.5$	0.328	0.316	0.086	0.081
		$n_1 = 25$	$\rho = 0$	0.280	0.249	0.083	0.079
		$n_2 = 75$	$\rho = 0.5$	0.277	0.240	0.083	0.071
Group covariance matrices differ	$p = 5$	$n_1 = 50$	$\rho = 0$	0.496	0.321	0.471	0.106
		$n_2 = 50$	$\rho = 0.5$	0.491	0.275	0.461	0.092
		$n_1 = 100$	$\rho = 0$	0.497	0.310	0.475	0.102
		$n_2 = 100$	$\rho = 0.5$	0.494	0.256	0.469	0.076
		$n_1 = 25$	$\rho = 0$	0.320	0.255	0.327	0.107
		$n_2 = 75$	$\rho = 0.5$	0.325	0.237	0.321	0.095
	$p = 10$	$n_1 = 50$	$\rho = 0$	0.492	0.242	0.458	0.044
		$n_2 = 50$	$\rho = 0.5$	0.489	0.177	0.438	0.034
		$n_1 = 100$	$\rho = 0$	0.495	0.234	0.467	0.041
		$n_2 = 100$	$\rho = 0.5$	0.490	0.160	0.455	0.029
		$n_1 = 25$	$\rho = 0$	0.372	0.216	0.395	0.040
		$n_2 = 75$	$\rho = 0.5$	0.381	0.164	0.403	0.043

Table 2. Average simulated test errors for LDA and KFDA in the case of lognormal populations

				Spacing 1		Spacing 2	
				LDA	KFDA	LDA	KFDA
Group mean vectors differ	$p = 5$	$n_1 = 50$	$\rho = 0$	0.290	0.161	0.099	0.053
		$n_2 = 50$	$\rho = 0.5$	0.244	0.120	0.060	0.049
		$n_1 = 100$	$\rho = 0$	0.286	0.129	0.098	0.040
		$n_2 = 100$	$\rho = 0.5$	0.243	0.097	0.055	0.036
		$n_1 = 25$	$\rho = 0$	0.166	0.135	0.052	0.051
		$n_2 = 75$	$\rho = 0.5$	0.094	0.089	0.035	0.038
	$p = 10$	$n_1 = 50$	$\rho = 0$	0.312	0.220	0.133	0.087
		$n_2 = 50$	$\rho = 0.5$	0.243	0.127	0.068	0.054
		$n_1 = 100$	$\rho = 0$	0.302	0.186	0.127	0.064
		$n_2 = 100$	$\rho = 0.5$	0.246	0.103	0.059	0.042
		$n_1 = 25$	$\rho = 0$	0.216	0.190	0.081	0.085
		$n_2 = 75$	$\rho = 0.5$	0.119	0.099	0.041	0.045
Group covariance matrices differ	$p = 5$	$n_1 = 50$	$\rho = 0$	0.278	0.281	0.115	0.056
		$n_2 = 50$	$\rho = 0.5$	0.362	0.349	0.209	0.144
		$n_1 = 100$	$\rho = 0$	0.272	0.264	0.114	0.046
		$n_2 = 100$	$\rho = 0.5$	0.357	0.345	0.208	0.139
		$n_1 = 25$	$\rho = 0$	0.213	0.209	0.083	0.047
		$n_2 = 75$	$\rho = 0.5$	0.256	0.254	0.142	0.117
	$p = 10$	$n_1 = 50$	$\rho = 0$	0.218	0.221	0.059	0.024
		$n_2 = 50$	$\rho = 0.5$	0.370	0.340	0.203	0.131
		$n_1 = 100$	$\rho = 0$	0.208	0.203	0.057	0.019
		$n_2 = 100$	$\rho = 0.5$	0.354	0.334	0.202	0.128
		$n_1 = 25$	$\rho = 0$	0.182	0.165	0.051	0.019
		$n_2 = 75$	$\rho = 0.5$	0.291	0.255	0.163	0.107

constant λ . These two classifiers were applied to large sets of test data cases generated from the same underlying distribution, and average test errors were calculated. The whole process was repeated 200 times (therefore, 200 different sets of training and test data were generated), and the results were averaged. Table 1 provides a summary of the average test errors obtained for data from normal populations and Table 2 the average test errors for data from lognormal populations. The standard errors of the averages in Table 1 varied from 0.0005 to 0.004, generally being in the vicinity of 0.001, while the standard errors for the entries in Table 2 were generally even smaller. Consider first the cases summarized in the top half of Table 1, corresponding to normal input vectors from populations with the same covariance matrix but different mean vectors. This is the ideal scenario for LDA, and one would expect LDA to outperform KFDA, but this is not always the case. In general, the test errors are very similar, but KFDA is consistently better in the mixed sample cases. When we have balanced sample sizes, LDA mostly performs slightly better when the input variables are uncorrelated, while the reverse is true for correlated input variables. Consider next the cases summarized in the bottom half of Table 1, corresponding to normal input vectors from populations with the same mean vector but different covariance matrices. Here KFDA consistently performs much better than LDA, and the advantage of using a classification rule which is non-linear in input space, is evident. The superiority of KFDA, which is substantial in all cases, increases in the presence of correlation amongst the input variables. Consider now the entries in the top half of Table 2, corresponding to lognormal input vectors from populations with the same covariance matrix but different mean vectors. Once again KFDA yields much lower error rates than LDA, except in the mixed sample cases at large spacing, where the two techniques perform very similarly. It is interesting to note that

for both techniques the introduction of correlation amongst the input variables leads to a reduction in test error. Finally, regarding the test errors in the bottom half of Table 2, the two classification rules perform comparably at small spacings, while KFDA becomes superior at larger spacings. In these cases the test errors increase in the presence of correlation.

5. Applications and avenues for further research

Although less well known than support vector machines, KFDA has already been applied in a variety of practical problems. We mention two interesting examples. In Harmsen and Pearlman (2003), KFDA is used in steganalysis to detect hidden information being transmitted over the internet in the form of jpeg images, while Liu et al. (2004) investigate the use of KFDA in face recognition. Apart from these somewhat exotic applications, it is clear from benchmark investigations reported in the literature (see for example Van Gestel et al., 2004), and the evidence presented in Section 4, that KFDA yields excellent results when applied in classification problems. Undoubtedly its use will increase as suitable software for its implementation becomes more freely available.

One of the reasons for the relatively slow rate at which KFDA is being absorbed into mainstream statistical techniques is the perception that successful application of KFDA requires considerable prior input from the user, for example as far as determining values for the hyperparameters is concerned. This is changing as more methods for estimating the hyperparameter values from the training data are introduced – see Pelckmans et al. (2004) for a recent reference on this topic. We feel that the method described in Section 3 also makes a definite contribution in this regard, thereby bringing KFDA closer to

being an off-the-shelf classification procedure (see Hastie et al., 2001, for an explanation of the term “off-the-shelf”).

There is much scope for further research into various aspects of KFDA. Although several contributions have appeared in the literature on quantifying the relative importance of the input variables in a kernel context (see for example Grandvalet and Canu, 2003), this is a difficult problem that has not yet been satisfactorily solved. Part of the difficulty lies in the fact that the summation in the KFDA classification rule (2.6) takes place over the data cases and not the input variables. Research into developing measures of variable importance in KFDA, and for kernel methods in general, is currently in progress. Further aspects that still require attention include the development of fast, generally applicable methods for determining the values of the hyperparameters, investigations into the sensitivity of KFDA (and other kernel methods) to outliers and accompanying remedial measures, and the investigation of various statistical inference problems within a kernel context.

Acknowledgement

The authors would like to thank Steve Kroon for helpful discussions and input during preparation of the paper, and a referee for comment which contributed to an improved paper.

References

- BOSER, B.E., GUYON, I.M. and VAPNIK, V.N. (1992). A training algorithm for optimal margin classifiers. In D. Haussler (Ed.), *5th Annual ACM Workshop on COLT*. Pittsburg, PA, ACM Press.
- CRISTIANINI, N., KANDOLA, J., ELISSEEFF, A. and SHAW-TAYLOR, J. (2002). On kernel-target alignment. In T. Dietterich, S. Becker and D. Cohn, (Eds.), *Advances in Neural Information Processing Systems*, **14**, MIT Press, Cambridge.

- GRANDVALET, Y. and CANU, S. (2003). Adaptive scaling for feature selection in SVMs. *Advances in Neural Information Processing Systems*, **15**, MIT Press, London.
- HARMSSEN, J.J. and PEARLMAN, W.A. (2003). Steganalysis of additive noise modelable information hiding. In B. Vasudev, T. R. Hsing, A.G. Tescher, T. Ebrahimi, (Eds.), *Proceedings of SPIE/IS&T Electronic Imaging*, **5022**, IS&T, Santa Clara, California.
- HASTIE, T., TIBSHIRANI, R. and FRIEDMAN, J. (2001). *The elements of statistical learning: data mining, inference, and prediction*. Springer, New York.
- HERBRICH, R. (2001). *Learning kernel classifiers*. MIT Press, London.
- KROON, S. and OMLIN, C.W. (2004a). Getting to grips with support vector machines: theory. Accepted for publication in the *South African Statistical Journal*.
- KROON, S. and OMLIN, C.W. (2004b). Getting to grips with support vector machines: application. Accepted for publication in the *South African Statistical Journal*.
- LIU, Q., LU, H. and MA, S. (2004). Improving kernel Fisher discriminant analysis for face recognition. *IEEE Transactions on Circuits and Systems for Video Technology*, **14**, 42–49.
- MIKA, S., RÄTSCH, G., WESTON, J., SCHÖLKOPF, B. and MÜLLER, K.-R. (1999). Fisher discriminant analysis with kernels. In Y.-H. Hu, J. Larsen, E. Wilson and S. Douglas (Eds.), *Neural Networks for Signal Processing*, **IX**, 41–48, IEEE Press, New York.
- MIKA, S., RÄTSCH, G. and MÜLLER, K.-R. (2001). A mathematical programming approach to the kernel Fisher algorithm. In T.K. Leen, T.G. Dietterich and V. Tresp (Eds.), *Advances in Neural Information Processing Systems*, **13**, 591–597, MIT Press, London.
- PELCKMANS, K., SUYKENS, J.A.K. and DE MOOR, B. (2004). Regularization constants in LS-SVMs: a fast estimate via convex optimization. Accepted for publication in the *Proceedings of the International Joint Conference on Neural Networks*.

- R: Software developed as part of the *R Project for Statistical Computing*. Available from the website <http://www.r-project.org/>
- SCHÖLKOPF, B. and SMOLA, A.J. (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT Press, London.
- SHAWE-TAYLOR, J. and CRISTIANINI, N. (2004). *Kernel methods for pattern analysis*. Cambridge University Press, Cambridge.
- STEEL, S.J. and LOUW, N. (2004). Simple and fast model selection for the Gaussian kernel. Submitted for publication to *Neurocomputing*.
- SUYKENS, J.A.K., VAN GESTEL, T., DE BRABANTER, J., DE MOOR, B. and VANDEWALLE, J. (2002). *Least squares support vector machines*. World Scientific Press, London.
- SUYKENS, J.A.K., DE MOOR, B. and VANDEWALLE, J. (2004). *LS-SVMlab: a toolbox for least squares support vector machines*. <http://www.esat.kuleuven.ac.be/sista/lssvmlab>.
- VAN GESTEL, T., SUYKENS, J.A.K., LANCKRIET, G., LAMBRECHTS, A., DE MOOR, B. and VANDEWALLE, J. (2002). Bayesian framework for least squares support vector machine classifiers, Gaussian processes and kernel Fisher discriminant analysis. *Neural Computation*, **14**, 1115–1147.
- VAN GESTEL, T., SUYKENS, J.A.K., BAESENS, B., VIAENE, S., VANTHIENEN, J., DEDENE, G., DE MOOR, B. and VANDEWALLE, J. (2004). Benchmarking least squares support vector machine classifiers. *Machine Learning*, **54**, 5–32.